

Teach Module 41: Containers and Docker Image Fundamentals

This guide was created from the chat session for Module 41. The course document was used only to identify the module topic. The teaching content below is original practice-oriented instruction.

Module Topic

Containers and Docker Image Fundamentals

Containers give a Java application a predictable runtime environment. Instead of depending on whatever Java version, settings, files, and tools happen to exist on a server, you package the application with the runtime expectations it needs.

A useful mental model:

```
Dockerfile -> Image -> Container
```

- A **Dockerfile** is the recipe.
- A **Docker image** is the packaged blueprint.
- A **container** is a running instance of that image.

For a Java service, the container can define:

```
Use Java 17
Copy the application JAR
Expose port 8080
Run java -jar app.jar
```

The main benefit is consistency:

```
Same image
Same startup command
Same runtime expectations
Different environments
```

1. Why Containers Matter

Without containers, deployment problems often sound like:

```
It works on my machine.
```

That usually means the developer's machine has something the target environment does not have, such as:

- the right Java version
- environment variables
- config files
- filesystem paths
- dependency versions

- startup commands

Containers reduce that problem by making the runtime explicit.

Instead of manually installing Java and configuring the server every time, you build an image that contains the application and enough runtime setup to start it reliably.

2. Dockerfile Basics

A `Dockerfile` is a plain text file containing instructions for building a Docker image.

Simple Java example:

```
FROM eclipse-temurin:17-jre

WORKDIR /app

COPY target/my-app.jar app.jar

EXPOSE 8080

CMD ["java", "-jar", "app.jar"]
```

Line by line:

```
FROM eclipse-temurin:17-jre
```

Start from an existing image that already has Java installed.

```
WORKDIR /app
```

Set `/app` as the working folder inside the container.

```
COPY target/my-app.jar app.jar
```

Copy the built Java application into the image.

```
EXPOSE 8080
```

Document that the application listens on port `8080`.

```
CMD ["java", "-jar", "app.jar"]
```

Tell the container what command to run when it starts.

3. Images And Containers

An image is not the running application. It is the template used to create the running application.

```
Image:      my-spring-api:1.0
Container:  the running API process created from that image
```

You can run multiple containers from the same image:

```
docker run -d -p 8081:8080 --name app-one my-spring-api:1.0
docker run -d -p 8082:8080 --name app-two my-spring-api:1.0
```

Both containers use the same image, but each container has its own running process, network mapping, name, and lifecycle.

4. Image Layers

Docker images are built in layers. Each Dockerfile instruction usually creates a new layer.

For example:

```
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY target/my-app.jar app.jar
CMD ["java", "-jar", "app.jar"]
```

Docker can reuse cached layers when nothing has changed. If only your JAR changes, Docker may reuse the base Java image layer instead of downloading it again.

This is one reason Docker builds are often faster after the first build.

Useful command:

```
docker history my-spring-api:1.0
```

This shows the layers that make up an image.

5. Build And Run Commands

Build an image:

```
docker build -t my-java-api:1.0 .
```

Meaning:

<code>docker build</code>	Build an image
<code>-t my-java-api:1.0</code>	Name and tag the image
<code>.</code>	Use the current folder as the build context

Run a container:

```
docker run -p 8080:8080 my-java-api:1.0
```

Meaning:

```
-p 8080:8080
```

Map port `8080` on your computer to port `8080` inside the container.

Then the app may be reachable at:

```
http://localhost:8080
```

6. Tags And Versioning

Image tags identify versions of an image.

Example:

```
my-company/payment-service:1.4.2
```

Breakdown:

my-company/payment-service	Image name
1.4.2	Tag/version

Avoid relying only on:

```
latest
```

The `latest` tag can change over time and make deployments unclear.

Better tags:

```
payment-service:1.0.0  
payment-service:2026-08-02  
payment-service:commit-a91f3c2
```

In CI/CD, commit-based tags are especially useful because they connect an image back to the exact code that produced it.

7. Container Registries

A registry stores Docker images so teams and deployment environments can pull them.

Common examples:

- Docker Hub
- Amazon ECR
- Azure Container Registry
- Google Artifact Registry
- GitHub Container Registry

- Red Hat Quay

Typical workflow:

```
docker build -t payment-service:1.0 .
docker tag payment-service:1.0 registry.example.com/payment-service:1.0
docker push registry.example.com/payment-service:1.0
```

Then another machine or deployment platform can pull it:

```
docker pull registry.example.com/payment-service:1.0
```

8. Runtime Configuration With Environment Variables

Containers should not hard-code environment-specific settings.

Bad idea:

```
database.url=prod-db.company.com
```

Better idea:

```
database.url=${DATABASE_URL}
```

Run the container with:

```
docker run -e DATABASE_URL=jdbc:postgresql://localhost:5432/appdb my-java-api:1.0
```

For Spring Boot, this pattern is common:

```
docker run \
-e SPRING_PROFILES_ACTIVE=dev \
-e DB_HOST=localhost \
-p 8080:8080 \
my-java-api:1.0
```

The same image can run in development, testing, staging, and production with different environment variables.

9. Container Image Security Basics

Container security starts with the image.

Good practices:

- Use trusted base images.
- Use smaller runtime images when possible.
- Do not put passwords or secrets into images.
- Keep base images updated.
- Run vulnerability scans.

- Avoid running as root when possible.
- Only copy what the application needs.

Example using a non-root user:

```
FROM eclipse-temurin:17-jre

WORKDIR /app

RUN useradd appuser

COPY target/my-app.jar app.jar

USER appuser

EXPOSE 8080

CMD ["java", "-jar", "app.jar"]
```

If an attacker compromises the application process, running as a low-privilege user limits damage.

10. Multi-Stage Docker Builds

A common Java container mistake is leaving build tools, source code, and unnecessary files inside the final runtime image.

A better pattern is a multi-stage build:

```
FROM maven:3.9-eclipse-temurin-17 AS build

WORKDIR /src

COPY pom.xml .
COPY src ./src

RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre

WORKDIR /app

COPY --from=build /src/target/*.jar app.jar

EXPOSE 8080

CMD ["java", "-jar", "app.jar"]
```

The first stage uses Maven to build the application. The second stage copies only the final JAR into a smaller runtime image.

Practice Exercises

Exercise 1: Containerize A Java App

Take any simple Java or Spring Boot app and create a `Dockerfile` .

Goal:

```
Build a Docker image
Run it as a container
Access the app from localhost
```

Practice commands:

```
docker build -t hello-java:1.0 .
docker run -p 8080:8080 hello-java:1.0
```

Success check:

```
The app starts inside Docker and responds at http://localhost:8080
```

Exercise 2: Compare Image And Container

Build one image, then run multiple containers from it:

```
docker run -d -p 8081:8080 --name app-one hello-java:1.0
docker run -d -p 8082:8080 --name app-two hello-java:1.0
```

Inspect:

```
docker images
docker ps
```

Practice questions:

```
What is shared by both containers?
What is unique to each running container?
```

Exercise 3: Use Environment Variables

Modify the Java app so it reads a value from an environment variable, such as:

```
APP_MESSAGE
```

Run it with:

```
docker run -p 8080:8080 -e APP_MESSAGE="Hello from Docker" hello-java:1.0
```

Success check:

The app displays or logs the value from APP_MESSAGE.

Exercise 4: Create Better Image Tags

Build the same app with different tags:

```
docker build -t hello-java:1.0 .
docker build -t hello-java:dev .
docker build -t hello-java:2026-08-02 .
```

Then list images:

```
docker images
```

Practice question:

Why is hello-java:1.0 clearer than hello-java:latest?

Exercise 5: Multi-Stage Docker Build

Create a Dockerfile that builds the app using Maven in one stage, then runs it in a smaller Java runtime image.

```
FROM maven:3.9-eclipse-temurin-17 AS build

WORKDIR /src

COPY pom.xml .
COPY src ./src

RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre

WORKDIR /app

COPY --from=build /src/target/*.jar app.jar

EXPOSE 8080

CMD ["java", "-jar", "app.jar"]
```

Success check:

The final image runs without Maven being installed inside the runtime container.

Exercise 6: Inspect Image Layers

Build an image and inspect it:

```
docker history hello-java:1.0
docker inspect hello-java:1.0
```

Practice questions:

Which Dockerfile instruction created each layer?
What information appears in docker inspect?

Exercise 7: Push To A Registry

Use Docker Hub, GitHub Container Registry, or a local registry.

Example flow:

```
docker tag hello-java:1.0 your-username/hello-java:1.0
docker push your-username/hello-java:1.0
```

Then pull it back:

```
docker pull your-username/hello-java:1.0
```

Success check:

The image can be pulled and run on another machine or environment.

Exercise 8: Avoid Secrets In Images

Create a fake secret file:

```
secret.txt
```

Then accidentally copy it into an image. Use this to learn what not to do.

Inspect the image or container and verify the problem. Then fix it using `.dockerignore`.

Example `.dockerignore` :

```
secret.txt
.git
*.log
```

Practice question:

Why should secrets be passed at runtime instead of baked into images?

Exercise 9: Run As Non-Root User

Update the Dockerfile to create and use a non-root user:

```
RUN useradd appuser
USER appuser
```

Then run:

```
docker run hello-java:1.0
```

Success check:

The application still runs, but not as root.

Exercise 10: Debug A Broken Container

Intentionally break the startup command:

```
CMD ["java", "-jar", "wrong-name.jar"]
```

Build and run it.

Then debug with:

```
docker ps -a
docker logs <container-name-or-id>
```

Practice goal:

Use Docker logs to identify why the container exited.

Lab: Containerize A Java Application

Goal

Build a Docker image for a Java application, run it as a container, configure it with environment variables, and inspect/debug it.

Prerequisites

You should have:

Docker installed
Java project or Spring Boot project
Terminal access

You can use any simple Java or Spring Boot app. The app should produce a `.jar` file.

Part 1: Create A Simple Java App

If you already have a Java app, skip this part.

Create a simple Spring Boot REST endpoint:

```
@RestController
public class HelloController {

    @Value("${APP_MESSAGE:Hello from default config}")
    private String message;

    @GetMapping("/")
    public String hello() {
        return message;
    }
}
```

Build the app:

```
mvn clean package
```

You should get something like:

```
target/my-app.jar
```

Part 2: Create A Dockerfile

In the project root, create:

```
Dockerfile
```

Add:

```
FROM eclipse-temurin:17-jre

WORKDIR /app

COPY target/*.jar app.jar

EXPOSE 8080
```

```
CMD ["java", "-jar", "app.jar"]
```

This:

```
Uses Java 17 runtime  
Creates /app inside the image  
Copies the built JAR  
Documents port 8080  
Starts the app
```

Part 3: Build The Docker Image

Run:

```
docker build -t java-container-lab:1.0 .
```

Verify:

```
docker images
```

You should see:

```
java-container-lab  1.0
```

Part 4: Run The Container

Run:

```
docker run -p 8080:8080 java-container-lab:1.0
```

Open:

```
http://localhost:8080
```

Expected result:

```
Hello from default config
```

Stop the container with `Ctrl + C` .

Part 5: Run With Environment Variables

Run:

```
docker run -p 8080:8080 -e APP_MESSAGE="Hello from Docker config" java-container-lab:1.0
```

Open:

```
http://localhost:8080
```

Expected result:

```
Hello from Docker config
```

This proves the same image can behave differently based on runtime configuration.

Part 6: Run In Detached Mode

Run:

```
docker run -d -p 8080:8080 --name java-lab java-container-lab:1.0
```

Check running containers:

```
docker ps
```

View logs:

```
docker logs java-lab
```

Stop it:

```
docker stop java-lab
```

Remove it:

```
docker rm java-lab
```

Part 7: Add A .dockerignore File

Create:

```
.dockerignore
```

Add:

```
.git  
*.log  
secret.txt
```

If your Dockerfile uses `COPY target/*.jar app.jar`, do not ignore `target/` yet. If you later switch to a multi-stage build that builds the app inside Docker, ignoring `target/` is usually fine.

Part 8: Create A Multi-Stage Dockerfile

Replace the Dockerfile with:

```
FROM maven:3.9-eclipse-temurin-17 AS build

WORKDIR /src

COPY pom.xml .
COPY src ./src

RUN mvn clean package -DskipTests

FROM eclipse-temurin:17-jre

WORKDIR /app

COPY --from=build /src/target/*.jar app.jar

EXPOSE 8080

CMD ["java", "-jar", "app.jar"]
```

Now the image builds the app inside Docker.

Build:

```
docker build -t java-container-lab:2.0 .
```

Run:

```
docker run -p 8080:8080 java-container-lab:2.0
```

Part 9: Inspect Image Layers

Run:

```
docker history java-container-lab:2.0
```

Then:

```
docker inspect java-container-lab:2.0
```

Answer these:

```
Which base image is used?
What command starts the container?
```

What port is exposed?
How many layers do you see?

Part 10: Improve Security

Update the final stage:

```
FROM eclipse-temurin:17-jre

WORKDIR /app

RUN useradd appuser

COPY --from=build /src/target/*.jar app.jar

USER appuser

EXPOSE 8080

CMD ["java", "-jar", "app.jar"]
```

Build:

```
docker build -t java-container-lab:secure .
```

Run:

```
docker run -p 8080:8080 java-container-lab:secure
```

Success condition:

The app still runs, but the process is not running as root.

Part 11: Debug A Broken Container

Break the Dockerfile intentionally:

```
CMD ["java", "-jar", "missing.jar"]
```

Build:

```
docker build -t java-container-lab:broken .
```

Run:

```
docker run --name broken-lab java-container-lab:broken
```

Check exited containers:

```
docker ps -a
```

View logs:

```
docker logs broken-lab
```

Expected error:

```
Unable to access jarfile missing.jar
```

Clean up:

```
docker rm broken-lab
```

Then fix the Dockerfile back to:

```
CMD ["java", "-jar", "app.jar"]
```

Lab Deliverables

At the end, you should be able to show:

```
Dockerfile
.dockerignore
Successful docker build output
Successful docker run output
Screenshot or curl output from localhost:8080
docker images output
docker ps output
docker logs output
docker history output
```

Knowledge Check

Answer these after the lab:

1. What is the difference between an image and a container?
2. Why do we use image tags?
3. Why should configuration come from environment variables?
4. Why should secrets not be copied into an image?
5. What is the benefit of a multi-stage Dockerfile?
6. Why is running as a non-root user better?

Quick Summary

For Module 41, remember:

```
A Dockerfile creates an image.  
An image is a packaged blueprint.  
A container is a running instance of that image.  
Tags identify image versions.  
Registries store and distribute images.  
Environment variables configure containers at runtime.  
Secure images should be minimal, updated, scanned, and avoid secrets.
```

The most important idea:

Containers make application runtime environments repeatable, portable, and easier to deploy.